

1. *Cotação de cada pergunta:* 1. 32 / 2. 36 / 3. 32 (Total: 100 pontos)
2. Responda às questões de forma clara e concisa nas folhas de exame distribuídas.

1. (Valorização: 32%) Responda às seguintes questões:

a) Faça o estudo da complexidade do método `f()` descrito a seguir:

```
int f(int n)
    int count = 0;
    int j = n;
    while (j>0) {
        j--;
        for (int k=0; k<n/2; k++)
            count++;
    }
    return count;
}
```

- b) Indique e justifique qual é a complexidade assintótica de um algoritmo de ordenação de n elementos que usa uma heap do seguinte modo: começando com uma heap vazia, cada elemento é adicionado um a um à heap; depois de todos os elementos estarem na heap, cada elemento é retirado um a um da heap.
- c) Considere que existem dois vectores `a[]` e `b[]` com n inteiros, sem números repetidos e ordenados por ordem estritamente crescente. Escreva um método em Java, `uniaoOrdenada(int n, int a[], int b[], int c[])`, em que dados os vectores `a[]` e `b[]` de dimensão n produza como resultado um vector `c[]` de dimensão $2*n$ com os elementos de `a[]` e `b[]` ordenados por ordem crescente. Indique, justificando, qual a complexidade do seu método. Para ter o máximo da pontuação, o seu método deve ter complexidade linear, ou seja, $\mathcal{O}(n)$.
- d) Considere que recebe os seguintes números por esta ordem: 5 8 1 6 3 2 7. Suponha que vai inserir os elementos numa heap, um a um, mantendo sempre a condição de heap. Desenhe o estado da heap depois de cada inserção.

2. (Valorização: 36%) Listas

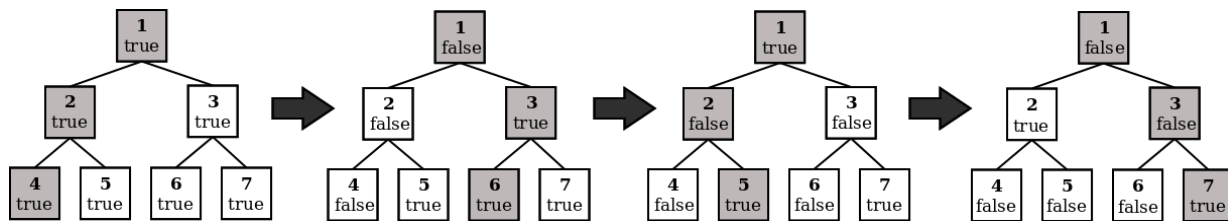
Num concurso de programação existem várias equipas a tentar resolver np problemas, numerados de 0 a $np - 1$. Cada **Equipa** é identificada pelo seu nome, universidade e país. Ao longo do concurso as equipas submetem código para tentar resolver um problema, com estas tentativas a serem chamadas de submissões. Cada **Submissao** é identificada pela equipa que a fez, o número do problema que estão a tentar resolver, o tempo da submissão (em minutos) e o resultado da submissão (correcta ou errada). Pode assumir que cada equipa terá no máximo uma submissão certa por problema.

No final a classificação é feita tendo em conta quantos problemas cada equipa resolveu, e quanto tempo gastou. Para calcular o tempo gasto de uma equipa, deve somar-se o tempo de cada uma das suas submissões correctas, acrescido de 20 minutos de penalização por cada submissão errada num problema que a equipa acertou (se a equipa não chegar a acertar o problema, as submissões erradas não penalizam).

- a) Escreva as classes **Equipa** e **Submissao**, necessárias para representar a informação, incluindo pelo menos um construtor para cada uma delas.
- b) Para gerir o concurso, será usada uma fila de submissões, ordenadas de forma crescente pelo tempo. Escreva uma classe **Fila<E>** que implementa filas com genéricos. Escreva os atributos, construtor(es), e apenas as assinaturas dos métodos habituais. Implemente ainda o método para retirar o primeiro elemento da fila. *Nota:* pode também apresentar uma implementação sem genéricos, mas terá uma penalização de 15%.
- c) Escreva um método **void atualizaPontuacoes(Fila<Submissao> f)**, parte duma classe principal **Concurso**, que será invocado no final para processar as submissões. Este método deverá percorrer a lista de submissões e deve calcular para cada equipa o número de problemas resolvidos e o tempo total gasto pela equipa, tal como atrás descrito. *Nota:* implemente um método **void pontuacao(int problema, int tempo, boolean acertou)** na classe **Equipa** que permite atualizar o número de tentativas erradas da equipa para um dado problema, se **acertou** for **false**, ou atualizar o número de problemas resolvidos e o tempo gasto, se **acertou** for **true**.
- d) Para ordenar no final as equipas vai-se usar o método **sort()** da API do Java. Para isso é necessário implementar o método **int compareTo(Equipa outra)** na classe **Equipa**. Escreva uma implementação para este método, considerando que a equipa em primeiro lugar deve ser a que tenha maior número de problemas resolvidos e em caso de empate deve vir primeiro a equipa que gastou menos tempo. Se ainda assim se verificar um empate, devem vir por ordem alfabética do seu nome.

3. (Valorização: 32%) Árvores

Considerem-se árvores binárias completas em que cada nó tem, além das ligações para os filhos esquerdo e direito, um valor inteiro positivo e um valor booleano. Os números inteiros são consecutivos e dados de cima para baixo, da esquerda para a direita. Os valores booleanos indicam o que fazer num percurso de descida: se o valor for **true** segue-se pela esquerda, se for **false**, segue-se pela direita. Inicialmente todos os booleanos estão colocados **true** e de cada vez que se percorre a árvore, muda-se o valor dos nós percorridos para o valor inverso. A figura seguinte ilustra uma árvore com 7 nós e o que acontece nos primeiros 4 percursos (indicados a cinzento), que terminam respectivamente nas folhas com números 4, 6, 5 e 7.



- Defina árvore binária completa e desenvolva estruturas de dados necessárias (classes) que implementem o tipo de árvores necessárias para este problema, incluindo todos os atributos necessários e pelo menos um construtor por classe.
- Escreva um método `void criaArvore(int n)`, da classe que implementa a árvore, que, começando na raiz, cria uma árvore completa com `n` níveis como atrás descrito, com os números preenchidos e todos os valores booleanos como **true**.
- Escreva um método `int ondeCai(int k)` que diz em que nó folha (valor inteiro do nó) termina o `k`-ésimo percurso a partir da raiz.